

Wumpus World Database Management System

Group 01

A Full-Stack Operational Database & Simulation-Tracking Platform for AI Agent Performance Analysis

Course: Database and Data Warehousing

Group members:

Mr. Aditya Bakhale(27PGAI0125)

Ms. Aishwarya Nevrekar(27PGAI0028)

Mr. Ashutosh Bansal(27PGAI0077)

Mr. RaghavendraPrasad Shetti(27PGAI0052)

1. Abstract

The Wumpus World is a classic Artificial Intelligence environment used to demonstrate knowledge-based agents, logical inference, and decision-making under partial observability. Most textbook implementations run the simulation once and discard the outcome, which makes it impossible to study an agent's behaviour across many attempts.

This project addresses that gap by building a full-stack Database Management System that records, stores, and analyzes Wumpus World game sessions at a granular level — every move, percept, and score change. The system is built on PostgreSQL, a Flask (Python) backend following the Model-View-Controller pattern, and a Bootstrap-based web interface, and it lets a researcher register Worlds and Agents, generate Game Sessions, and query the resulting data to compare agents, identify the hardest worlds, and replay a session step-by-step on an interactive grid.

2. Introduction

2.1 Background

Traditional Wumpus World implementations focus entirely on the agent's reasoning logic — a single run executes and the result is discarded. Studying long-term performance, comparing algorithms, and finding statistical patterns in how and why agents fail requires a persistent, structured store of session history. This project bridges classical AI theory with database engineering and data-warehousing practice.

2.2 Objectives

- Design a normalized, scalable relational schema covering users, worlds, agents, sessions, steps, and percepts.
- Build a web interface for creating and managing Worlds, Agents, Users, and Game Sessions (full CRUD).
- Enforce official Wumpus World rules and scoring logic automatically wherever data is written.
- Record step-level data — position, action, direction, and percepts felt — for every session.
- Provide analytical SQL for win rates, cell-visit heatmaps, score progression, and agent comparison.
- Reconstruct and visually replay a stored session on an interactive grid in the browser.

2.3 Scope

- Users — human researchers/operators who register worlds, agents, and initiate sessions.
- Agents — the AI algorithms or strategies being evaluated (Human, Random, Rule-Based, Logic-Based).
- Worlds — configurable grid layouts (row/column dimensions, difficulty, hazard and object placement).
- Game Sessions — one full attempt by an Agent on a World, with outcome, score, and timing.
- Game Steps & Perceptions — the turn-by-turn move history, including Stench/Breeze/Glitter/Bump/Scream.

3. System Analysis

3.1 Problem Statement

Evaluating AI agents requires historical evidence, not just a final pass/fail result. Without a structured database, information about how an agent dies, which paths it takes, and how efficiently it scores is lost the moment a run ends. The project's goal is to design a normalized relational database, paired with a web interface, that can reliably ingest, validate, and store this simulation data for later analysis.

3.2 Proposed System

A Flask web application backed by PostgreSQL where a user creates Worlds, Agents, and Users, then generates Game Sessions. The backend calculates session outcomes and scores using the official Wumpus World performance measures before persisting the record, and exposes an API endpoint that reconstructs a session's step history for a visual replay.

3.3 Functional Requirements

- Create, read, update, and delete Users, Worlds, Agents, and Game Sessions through the web UI.
- Automatically generate a plausible outcome (status, result, score, steps) for a new Game Session.
- Reject data that violates Wumpus World logic (e.g. a 'Running' session with an end time).
- Reconstruct and animate a session's move history from the database via a JSON API.

3.4 Non-Functional Requirements

- Scalability — the game_steps and step_perceptions tables are designed to hold very high row counts efficiently, with indexes on all foreign keys and frequently filtered columns.
- Data integrity — CHECK, UNIQUE, and FOREIGN KEY constraints enforce business rules at the database layer, not just in application code.
- Usability — a responsive Bootstrap 5 interface usable on both desktop and mobile.

4. Wumpus World Domain (The Rules)

The Wumpus World is a grid-based cave (a 4×4 layout in the classic formulation, though this system supports any grid from 2×2 up to 50×50) where the agent starts in the bottom-left cell. The goal is to find the gold, grab it, and return to the start to climb out.

4.1 Hazards

- Wumpus — a monster occupying one room; entering it kills the agent. It can be killed with a single arrow.
- Pits — bottomless holes; entering one is immediately fatal. A world may contain multiple pits.

4.2 Percepts

The agent never sees a hazard directly and must infer safety from indirect clues sensed in its current cell:

- Stench — the Wumpus occupies an adjacent cell.

- Breeze — a pit occupies an adjacent cell.
- Glitter — the gold is in the current cell.
- Bump — the agent attempted to move into a wall.
- Scream — the Wumpus has just been killed.

4.3 Standard Performance Measure

Scores are computed automatically according to the official rules:

Event	Score Impact
Find the gold and climb out	+1000
Die (Wumpus or pit)	-1000
Every action taken	-1 per step
Shoot an arrow	-10

5. System Design & Architecture

5.1 Technology Stack

Layer	Technology	Purpose
Database	PostgreSQL (psycopg v3, binary adapter)	Enterprise-grade RDBMS with strict CHECK constraints and rich SQL feature support
Backend framework	Python 3 + Flask 3.0.3	WSGI routing and HTTP request handling
ORM	Flask-SQLAlchemy 3.1.1	Maps Python classes to PostgreSQL tables; parameterized queries prevent SQL injection
Frontend	Bootstrap 5, Jinja2, vanilla JavaScript	Server-side rendered, responsive UI plus a canvas/DOM-based grid replay
Config	python-dotenv	Loads DATABASE_URL and SECRET_KEY from a local .env file

5.2 Architectural Pattern

The project follows the Model-View-Controller (MVC) pattern adapted for Flask via Blueprints: routes/modules act as controllers, models.py holds the SQLAlchemy ORM classes, and templates/ holds the Jinja2 views. Five blueprints are registered — users, worlds, agents, game_sessions, and simulation — each owning its own URL prefix and templates.

Directory Structure

```

MAIN_PROJECT/
├── schema.sql      # PostgreSQL DDL: tables, constraints, indexes
├── seed_data.sql   # Sample data for all 15 tables

```

```

├── queries.sql      # 40 annotated analytical/demo SQL queries
├── backend/
│   ├── app.py      # App factory; registers blueprints, dashboard route
│   ├── config.py   # Loads DATABASE_URL / SECRET_KEY from .env
│   ├── database.py  # SQLAlchemy instance + connection health check
│   ├── models.py   # ORM classes for all 9 SQLAlchemy-mapped tables
│   ├── requirements.txt # Flask, Flask-SQLAlchemy, psycopg[binary], dotenv
│   └── routes/
│       ├── users.py      # CRUD for Users
│       ├── worlds.py     # CRUD for Worlds
│       ├── agents.py     # CRUD for Agents
│       ├── game_sessions.py # CRUD + outcome/score generation engine
│       └── simulation.py  # Replay page + /api/simulation/<id> JSON feed
├── static/{css,js}  # Styling and the grid-replay animation engine
└── templates/       # base.html, dashboard, and per-entity form/list views

```

Data Flow

- The user submits an action (e.g. 'Create Session') from the Bootstrap UI.
- A Flask Blueprint route validates the submitted form and, for sessions, invokes the outcome-generation engine.
- SQLAlchemy translates the validated Python object into an INSERT/UPDATE, which PostgreSQL checks against its CHECK/FK constraints.
- On success, the controller re-queries the updated records and renders them back through a Jinja2 template.

6. Database Design

schema.sql defines a 3NF-normalized PostgreSQL schema of 15 tables, considerably more granular than a minimal 5-table design: it separates lookup/reference data (roles, hazard_types, object_types, action_types, perception_types) from transactional data (users, worlds, cells, agents, game_sessions, game_steps) and junction tables (user_roles, cell_hazards, cell_objects, step_perceptions).

6.1 Core Tables

Table	Purpose	Key Constraints
users	Registered researchers/operators	Unique email; regex-validated email format; case-insensitive unique index
roles / user_roles	Role-based access (Admin, Instructor, Student, Viewer)	Many-to-many junction with cascading delete on user_roles
worlds	Reusable grid layout definitions	grid_rows/columns between 2–50; difficulty in {Easy, Medium, Hard, Custom}
cells	Every (row, column) coordinate for a world	Unique per (world, row, col); at most one start cell per world (partial unique index)
hazard_types / cell_hazards	Pit / Wumpus placement per cell	Unique (cell, hazard type) pairing

Table	Purpose	Key Constraints
object_types / cell_objects	Gold / Exit placement per cell	Tracks is_collected status
agents	AI strategy profiles under test	agent_type restricted to Human, Random, Rule-Based, Logic-Based
game_sessions	One full attempt by an agent on a world	status/result CHECK lists; end_time required unless status = Running
action_types	Valid agent actions (Move, Turn, Shoot, Grab, Climb ...)	Unique action_name
game_steps	Turn-by-turn move history for a session	Unique (session, step_number); direction restricted to N/S/E/W
perception_types / step_perceptions	Percepts felt on each step	Many-to-many junction between steps and percept types

6.2 Referential Integrity

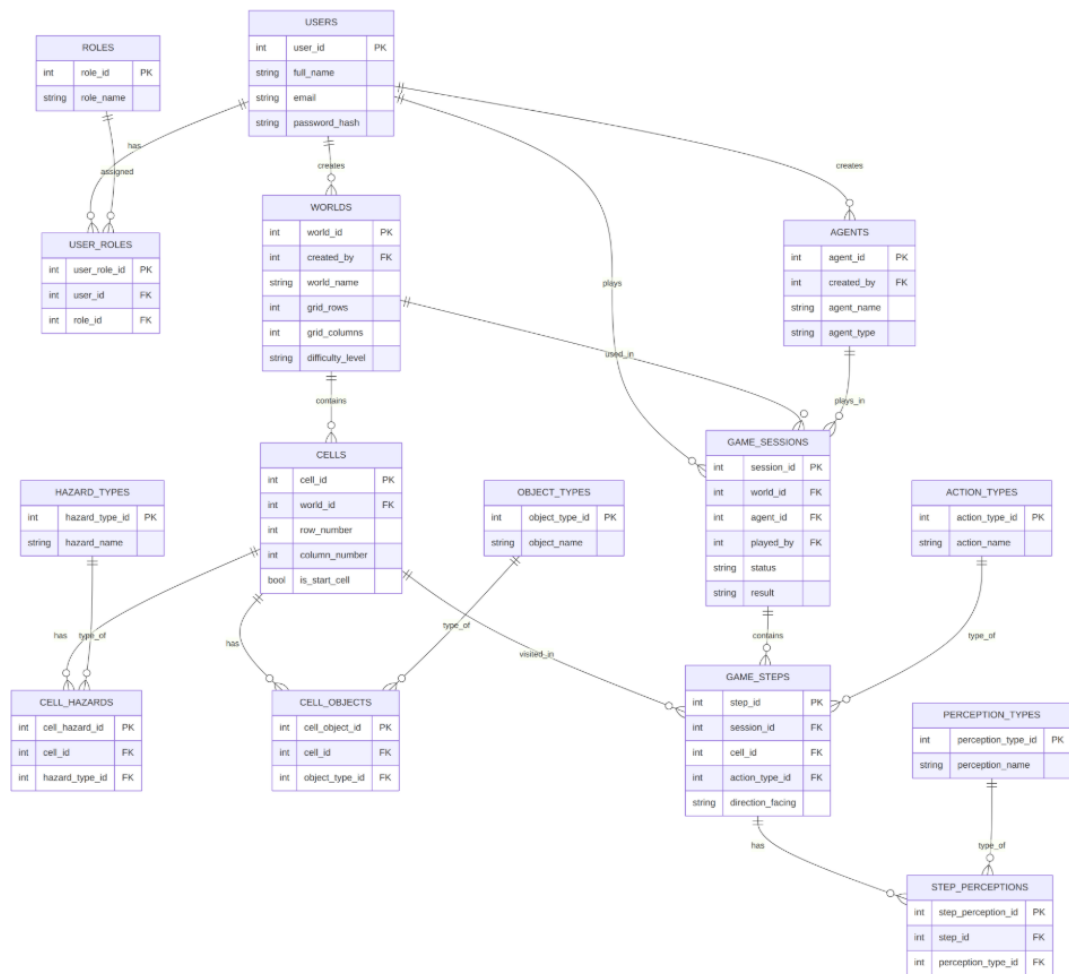
Foreign keys consistently use ON UPDATE CASCADE. Deletes are RESTRICT for reference/master data (e.g. a World cannot be deleted while sessions reference it) and CASCADE for dependent detail rows (deleting a session removes its steps; deleting a step removes its perceptions).

6.3 Example Constraint — Session Consistency

This constraint is representative of how the schema pushes business rules down to the database layer rather than trusting the application alone:

```
CONSTRAINT chk_game_sessions_completed_has_end_time CHECK (
  (status = 'Running' AND end_time IS NULL)
  OR (status <> 'Running' AND end_time IS NOT NULL)
)
```

6.4 Schema Diagram



7. Implementation Details

7.1 Automated Outcome & Scoring Engine

When a session is created, the user only supplies a World, Agent, and Player. `routes/game_sessions.py` then simulates a plausible outcome using weighted randomization so that results vary realistically across many generated sessions:

```
STATUSES = ["Running", "Completed", "Failed", "Abandoned"]
# 10% Running, 40% Completed, 40% Failed, 10% Abandoned
data["status"] = random.choices(STATUSES, weights=[10, 40, 40, 10], k=1)[0]

if data["status"] == "Failed":
    data["result"] = random.choice([
        "Died (Fell in Pit)", "Died (Eaten by Wumpus)"
    ])

# Score = base outcome award - 1 per step - 10 per arrow shot
data["final_score"] = base_score - data["total_steps"] - (arrows_shot * 10)
```

The status determines the result category, the result determines the base score (+1000 / -1000 / 0), and the total step count (randomized between 5 and 300) and arrow usage are subtracted per the official performance measure — mirroring the scoring rules in Section 4.3 in code.

7.2 Full CRUD Coverage

Each of the four primary entities exposes a consistent set of routes:

Blueprint	URL prefix	Routes implemented
users	/users	list, new (create), edit, delete
worlds	/worlds	list, new (create), edit, delete
agents	/agents	list, new (create), edit, delete
game_sessions	/game-sessions	list, history (replay picker), new, edit, delete

7.3 Frontend Dynamic Behaviour

Vanilla JavaScript adjusts form constraints live — for example, marking the `end_time` field as required as soon as the session status is changed away from 'Running' — so invalid combinations are caught before the request ever reaches the server.

7.4 Interactive Session Replay

A notable implemented feature (beyond basic CRUD) is a step-by-step visual replay of a stored session. The `/api/simulation/<session_id>` endpoint joins `game_steps`, `cells`, `action_types`, and `step_perceptions` to reconstruct the full move history as JSON:

```
SELECT gs.step_number, c.row_number, c.column_number,  
       at.action_name, gs.direction_facing, gs.score_after_step,  
       STRING_AGG(pt.perception_name, ',') AS perceptions  
FROM game_steps gs  
JOIN cells c ON gs.cell_id = c.cell_id  
JOIN action_types at ON gs.action_type_id = at.action_type_id  
LEFT JOIN step_perceptions sp ON gs.step_id = sp.step_id  
LEFT JOIN perception_types pt ON sp.perception_type_id = pt.perception_type_id  
WHERE gs.session_id = :session_id  
GROUP BY gs.step_number, c.row_number, c.column_number,  
         at.action_name, gs.direction_facing, gs.score_after_step  
ORDER BY gs.step_number;
```

On the frontend, `static/js/simulation.js` consumes this feed and animates the agent moving across a rendered grid, updating score and percept indicators at each step. This turns the database from a passive log into a working playback tool.

8. Analytics & Data Warehouse Queries

queries.sql is a 40-query demonstration file covering the full breadth of SQL taught in the course — from basic filtering through joins, aggregation, subqueries, CTEs, window functions, views, transactions, stored procedures, and triggers — all written against this schema rather than toy tables.

#	Category	Example purpose
1–3	SELECT / WHERE / ORDER BY	List active users; find active Hard worlds; sort sessions by recency
4–6	INNER / LEFT / RIGHT JOIN	Session details with names; session counts per world (incl. zero); roles with/without users
7–9	GROUP BY / HAVING / Aggregates	Sessions, avg & best score per agent; agents with ≥ 2 sessions; overall summary stats
10–13	Subqueries	Above-average sessions; users with a win; per-world above-average scores; hazard-free cells
14–15	CTEs	Win rate per agent; most-visited cells across all sessions
16–17	Window functions	RANK() of sessions by score within a world; LAG()-based score-change trace
18–20	Multi-table joins	Step perceptions; world layout; optional cell objects
21–24	Views	Reusable session-summary and aggregate views
25–28	Transactions	INSERT / UPDATE / DELETE wrapped in explicit BEGIN...COMMIT
29–32	Functions & procedures	A PostgreSQL function, a stored procedure, and rollback demonstrations
33–35	Triggers	A trigger support function, the trigger itself, and a live demonstration
36–40	CASE / DISTINCT / LIMIT / EXISTS	Conditional labeling; distinct results; top-N; existence checks; a combined report query

Example — Win Rate per Agent (CTE)

```
WITH agent_results AS (
  SELECT agent_id, COUNT(*) AS total_sessions,
    COUNT(*) FILTER (WHERE result = 'Won') AS won_sessions
  FROM game_sessions GROUP BY agent_id
)
SELECT a.agent_name, ar.total_sessions, ar.won_sessions,
  ROUND((ar.won_sessions::NUMERIC / ar.total_sessions) * 100, 2) AS win_rate_percent
FROM agent_results ar
INNER JOIN agents a ON ar.agent_id = a.agent_id
ORDER BY win_rate_percent DESC;
```

9. Seed Data & Local Testing

seed_data.sql populates every table with representative rows so the analytical queries and the UI can be exercised immediately after schema creation:

Table	Seed rows
users	5
worlds	3
agents	3
game_sessions	5
cells	48
game_steps	82
step_perceptions	49
cell_hazards	12
cell_objects	6
user_roles	7
roles	4
action_types	7
perception_types	5
hazard_types	2
object_types	2

The backend also supports a zero-dependency SQLite fallback for quick local development: if DATABASE_URL points at a sqlite:/// file, app.py auto-creates the SQLAlchemy-mapped tables with db.create_all(). This mode was exercised locally (backend/instance/wumpus_local.db), where one user, one world, one agent, and one session were created through the live UI — confirming the create/list flow works end-to-end. Note that SQLite does not enforce the same CHECK constraints as PostgreSQL, so this mode is for UI/dev convenience only, not a substitute for testing against the real schema.

10. How to Run the Project

10.1 Database Setup (PostgreSQL)

```
createdb -U postgres wumpus_world_dbms
psql -U postgres -d wumpus_world_dbms -f schema.sql
psql -U postgres -d wumpus_world_dbms -f seed_data.sql
```

10.2 Backend Setup

```
cd backend
python3 -m venv .venv
source .venv/bin/activate    # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

10.3 Environment Configuration

Create backend/.env (or copy .env.example) with:

```
DATABASE_URL=postgresql+psycopg://postgres:postgres@localhost:5432/wumpus_world_dbms
SECRET_KEY=change-this-secret
```

10.4 Start the Application

```
python app.py
```

The application will be available at <http://127.0.0.1:5003>.

10.5 Quick Local Mode (no PostgreSQL)

For fast UI testing without installing PostgreSQL, set `DATABASE_URL=sqlite:///wumpus_local.db` in backend/.env and run `python app.py` — Flask-SQLAlchemy will auto-create the mapped tables. This mode skips schema.sql/seed_data.sql and does not enforce the PostgreSQL CHECK constraints described in Section 6.

11. Output (Snapshots):

Wumpus World DBMS

DashboardGame HistoryTheme

MAIN

Dashboard

Users

Worlds

Agents

Game Sessions

Game History

Dashboard

Database Connected

6

Users

4

Worlds

4

Agents

8

Sessions

3

Wins

Recent Sessions

View All

ID	PLAYER	WORLD	AGENT	RESULT	SCORE
11	Riya Sharma	Training Cave	Rule Scout	Won	805
7	Dr. Ananya Rao	Inference Cave	Logic Explorer	Escaped Without Gold	-189
6	Adi	Wonderland	Ashu Ka fanda	Incomplete	-137
5	Arjun Iyer	Expert Cave	Random Walker	Died	-1017
4	Riya Sharma	Inference Cave	Rule Scout	Escaped Without Gold	-16

Quick Actions

Add User

Create World

Create Agent

Start Session

Wumpus World DBMS

DashboardGame HistoryTheme

MAIN

Dashboard

Users

Worlds

Agents

Game Sessions

Game History

Users

New User

Create and maintain users who run simulations.

Search users...

ID	NAME	EMAIL	STATUS	CREATED	ACTIONS
1	Dr. Ananya Rao	ananya.rao@ji.edu	Active	2026-06-01	EditDelete
2	Prof. Vikram Mehta	vikram.mehta@ji.edu	Active	2026-06-01	EditDelete
3	Riya Sharma	riya.sharma@student.ji.edu	Active	2026-06-02	EditDelete
4	Arjun Iyer	arjun.iyer@student.ji.edu	Active	2026-06-02	EditDelete
5	Meera Nair	meera.nair@student.ji.edu	Active	2026-06-03	EditDelete
6	Adi	adi@gmail.com	Active	2026-07-03	EditDelete

Wumpus World DBMS Dashboard Game History Theme

MAIN
Dashboard
Users
Worlds
Agents
Game Sessions
Game History

Game Sessions

Manage playthrough summaries and outcomes.

[New Session](#)

Search sessions...

ID	WORLD	AGENT	PLAYER	STATUS	RESULT	SCORE	STEPS	ACTIONS		
11	Training Cave	Rule Scout	Riya Sharma	Completed	Won	805	185	History	Edit	Delete
7	Inference Cave	Logic Explorer	Dr. Ananya Rao	Completed	Escaped Without Gold	-189	179	History	Edit	Delete
6	Wonderland	Ashu Ka fanda	Adi	Abandoned	Incomplete	-137	137	History	Edit	Delete
5	Expert Cave	Random Walker	Arjun Iyer	Failed	Died	-1017	17	History	Edit	Delete
4	Inference Cave	Rule Scout	Riya Sharma	Completed	Escaped Without Gold	-16	16	History	Edit	Delete
3	Inference Cave	Logic Explorer	Meera Nair	Completed	Won	982	18	History	Edit	Delete

11. Conclusion & Future Scope

The Wumpus World Database Management System successfully connects classical AI theory with practical database engineering: a normalized 15-table PostgreSQL schema captures both high-level session outcomes and granular step-by-step percept/action data, a Flask MVC backend enforces the official scoring rules automatically, and a working replay tool turns stored history into an interactive visualization. Together with the 40-query analytical suite, the system provides a solid foundation for comparing AI agents, studying failure patterns, and practicing data-warehouse-style reporting on top of operational game data.

Future Enhancements

- Resolve the result-value CHECK constraint mismatch described in Section 10 before running further Failed-session tests against PostgreSQL.
- Connect a real Python-based Wumpus World agent to stream live game_steps into the database instead of the randomized outcome generator.
- Build a dedicated OLAP-style data warehouse schema (star schema with a session fact table) for larger-scale aggregate reporting.
- Add authentication so the existing roles/user_roles tables are actually enforced for access control.